

DOCUMENTAÇÃO DO PROJETO: AGENDA FLOW

Equipe: Caio Schneider, Gabriel de Lima, João Paulo, Pedro Oliveira

1. Objetivo do módulo

O projeto AgendaFlow tem como objetivo desenvolver um sistema web de agendamento e gestão de atendimentos.

O sistema permite que uma pessoa realize um agendamento online informando seus dados pessoais, escolhendo um espaço disponível, data, horário e status do atendimento. A aplicação também permite visualizar os agendamentos cadastrados, alterar o status e excluir registros.

O projeto foi dividido em duas partes principais:

Frontend: interface visual usada pelo usuário

Backend: API responsável pelas regras de negócio, banco de dados e integração externa

Essa divisão segue a proposta do roteiro da atividade, que pede separação entre front-end, back-end, banco de dados, integração por API e documentação do sistema.

2. Funcionalidades implementadas

Algumas funcionalidades que o projeto apresenta são:

- Cadastro de agendamento online
- Listagem de agendamentos
- Status dos agendamentos
- Alteração de status
- Exclusão de agendamento
- Gestão de espaços
- Integração com Google Agenda

3. Tecnologias utilizadas

Backend:

- Node.js
- Express.js
- MySQL
- mysql2
- CORS
- dotenv
- Nodemon
- Google Calendar API
- googleapis

- @google-cloud/local-auth

Frontend

- HTML5
- CSS3
- JavaScript
- Fetch API

Banco de dados

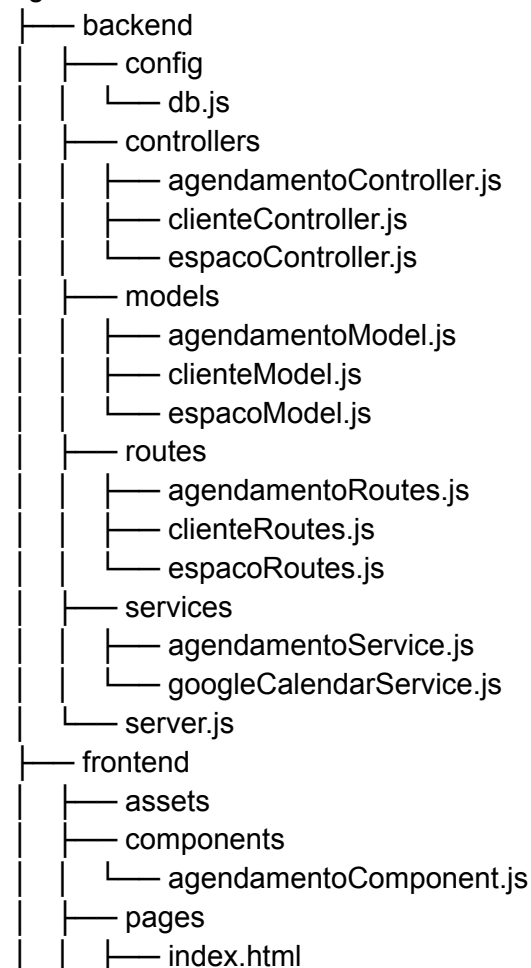
- MySQL
- MySQL Workbench

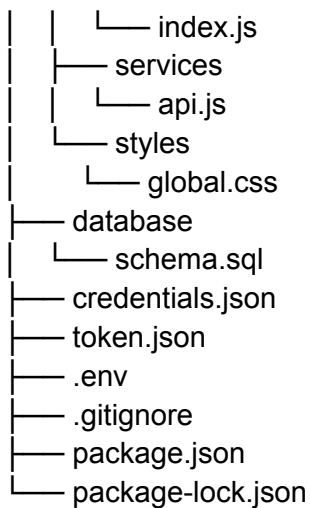
Ferramentas de desenvolvimento

- Visual Studio Code
- Node.js
- npm
- Git/GitHub
- Google Cloud Console

4. Estrutura de pastas do projeto

agenda-flow





5. Explicação das principais pastas

backend/config

- Contém arquivos de configuração do backend.
- O arquivo db.js é responsável por conectar o Node.js ao banco MySQL.

backend/models

- Os models são responsáveis por conversar diretamente com o banco de dados.
- Eles executam comandos SQL

backend/services

- Os services concentram as regras de negócio do sistema.

backend/controllers

- Os controllers recebem as requisições feitas pelo frontend, chamam os services ou models necessários e devolvem uma resposta para o usuário.

backend/routes

- As routes definem os caminhos da API.

frontend/pages

- Contém as páginas principais do frontend.
- O arquivo index.html representa a tela principal do sistema.

frontend/services

- Contém o arquivo api.js, responsável por fazer as requisições para o backend usando fetch.

frontend/components

- Contém componentes visuais reutilizáveis.
- O arquivo agendamentoComponent.js monta os cards de agendamento exibidos na tela.

frontend/styles

- Contém os arquivos CSS responsáveis pela aparência do sistema.

6. Estrutura do banco de dados

O banco utilizado no projeto se chama agenda_flow

Ele possui três tabelas principais:

- clientes
- espacos
- agendamentos

7. Endpoints da API

A API roda por padrão em: <http://localhost:3000>

7.1 Clientes

- Listar clientes
 - GET /api/clientes
 - Retorna todos os clientes cadastrados.
- Criar cliente
 - POST /api/clientes

7.2 Espaços

- Listar espaços
 - GET /api/espacos
 - Retorna todos os espaços ativos
- Criar espaço
 - POST /api/espacos

7.3 Agendamentos

- Listar agendamentos
 - GET /api/agendamentos
 - Retorna todos os agendamentos cadastrados, com dados do cliente e do espaço.
- Criar agendamento
 - POST /api/agendamentos
- Atualizar status do agendamento
 - PATCH /api/agendamentos/:id/status

- Excluir agendamento
 - DELETE /api/agendamentos/:id

8. Instruções para execução do projeto

8.1 Pré-requisitos

Antes de rodar o projeto, é necessário ter instalado:

- Node.js
- npm
- MySQL
- MySQL Workbench
- Visual Studio Code

8.2 Instalar dependências

Na raiz do projeto, execute:

```
npm install
```

8.3 Configurar o banco de dados

Abra o MySQL Workbench e execute o arquivo:

```
database/schema.sql
```

8.4 Configurar variáveis de ambiente

Na raiz do projeto, crie o arquivo .env com o seguinte conteúdo:

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=sua_senha_do_mysql  
DB_NAME=agenda_flow  
PORT=3000
```

Substitua sua_senha_do_mysql pela senha configurada no MySQL.

8.5 Configurar Google Agenda

Para usar a integração com o Google Agenda, é necessário:

Ativar a Google Calendar API no Google Cloud Console
Criar uma credencial OAuth do tipo App para computador
Baixar o arquivo JSON da credencial
Renomear o arquivo para credentials.json
Colocar o credentials.json na raiz do projeto
Adicionar o e-mail usado como usuário de teste na tela OAuth

A estrutura deve ficar assim:

```
agenda-flow
├── credentials.json
├── package.json
├── backend
├── frontend
└── database
```

Na primeira autorização bem-sucedida, será criado automaticamente o arquivo:

token.json

8.6 Rodar o backend

Na raiz do projeto, execute: `npm run dev`

Se funcionar corretamente, o terminal mostrará:

Servidor rodando em `http://localhost:3000`

8.7 Testar a API

Abra no navegador: `http://localhost:3000`

A resposta esperada é: API do AgendaFlow funcionando

Também é possível testar: `http://localhost:3000/api/agendamentos`

8.8 Rodar o frontend

Abra o arquivo: `frontend/pages/index.html`

O frontend se comunica com a API em: `http://localhost:3000/api`

Por isso, o backend precisa estar rodando antes.

9. Fluxo de funcionamento do sistema

1. O usuário abre a página do sistema
2. Preenche nome, email, telefone, espaço, data e horário
3. O frontend envia os dados para o backend
4. O backend valida os dados recebidos
5. O backend cria o cliente no banco
6. O backend cria o agendamento no banco
7. O backend tenta criar um evento no Google Agenda
8. O backend retorna uma resposta ao frontend
9. O frontend atualiza a lista de agendamentos na tela